TREINAMENTO DE BACK-END · CITI

🌿 Guia do desafio Folhagem & Cia

Conceitos, passo a passo, armadilhas e testes para construir a API da loja de plantas.

Node.js · Express · TypeScript · Prisma · PostgreSQL · Docker

Onde está cada coisa	
`DESAFIO.md`	O enunciado oficial: entidade `Planta`, rotas, regras de negócio, bônus, entregáveis e avaliação
`README.md`	Como instalar e rodar o projeto, comandos do dia a dia e erros de ambiente
Este guia	Como construir: conceitos, leitura do CRUD de usuário, passo a passo, testes e armadilhas

> 💡 **O segredo do desafio**
> O repositório já vem com um CRUD de usuários pronto e funcionando. Quase tudo que você precisa fazer nas plantas tem um equivalente nos usuários: abra os dois lado a lado e vá adaptando. Entender o exemplo vale mais do que copiá-lo.

Se você...	Leia
nunca fez uma API	O guia inteiro, na ordem
já sabe o que é uma API	Partes 2 (o projeto) e 3 (passo a passo)
travou em algum erro	Parte 5 (armadilhas) e os problemas comuns do README
está quase entregando	Parte 6 (antes de entregar)

1 · Conceitos básicos

O que é uma API?

Pense num restaurante. Você não entra na cozinha: faz o pedido ao garçom, que leva até a cozinha e volta com o prato.



A API recebe o pedido (a **requisição**), confere se faz sentido, busca ou grava no banco e devolve uma **resposta**. O cliente nunca fala direto com o banco.

CRUD

Letra	Operação	Método HTTP	Função do Prisma
C	Create (criar)	`POST`	`create`
R	Read (ler)	`GET`	`findMany`, `findUnique`
U	Update (atualizar)	`PATCH`	`update`
D	Delete (deletar)	`DELETE`	`delete`

> 📘 **PATCH ou PUT?**
> O PUT substitui o recurso inteiro. O PATCH altera só o que você mandar. Aqui usamos PATCH: se o corpo tiver só `{ "preco": 9990 }`, só o preço muda.

Anatomia de uma requisição

```
PATCH  http://localhost:3001/plantas/3?avisar=true
Content-Type: application/json

{ "preco": 9990 }
```

Onde	No exemplo	Como ler no Express	Para que serve
Params (parte da rota)	`3`	`req.params.id`	Identificar o recurso
Query string (depois do `?`)	`avisar=true`	`req.query.avisar`	Opções e filtros
Body (corpo)	`{ "preco": 9990 }`	`req.body.preco`	Os dados a criar ou mudar

> ⚠ **Params e query string são sempre texto**
> Em `/plantas/3`, `req.params.id` é a string `"3"`. Em `?petFriendly=true`, o valor é a string `"true"`. Você vai precisar converter. Já o body, por ser JSON, mantém os tipos.

Status HTTP

Status	Quando usar	Exemplo no desafio
`200 OK`	Deu certo e há algo para devolver	Listar, buscar, editar, vender
`201 Created`	Algo novo foi criado	Cadastrar planta
`204 No Content`	Deu certo e não há corpo	Deletar planta
`400 Bad Request`	O cliente mandou dados faltando ou inválidos	Preço negativo, venda sem estoque
`404 Not Found`	O recurso pedido não existe	Buscar a planta 999
`500 Internal Server Error`	Algo inesperado quebrou do nosso lado	O banco caiu

> 💡 **Regra de bolso**
> 2xx: deu certo. 4xx: a culpa é de quem pediu. 5xx: a culpa é nossa. Por isso o `catch` genérico responde 500, nunca 400.

JSON e tipos

```
{
  "nome": "Jiboia",          // string: entre aspas duplas
  "preco": 3490,             // número: SEM aspas
  "petFriendly": false,      // booleano: SEM aspas
  "luminosidade": "SOMBRA"   // enum: chega como string
}
```

`"preco": "3490"` é texto, não número, e uma API bem validada recusa com 400. JSON não aceita comentários: os `//` acima são só explicação.

`async` / `await` e `try` / `catch`

Falar com o banco leva tempo, então o Prisma devolve uma **Promise**. O `await` espera o resultado e só pode ser usado dentro de uma função `async`. O `try` / `catch` garante que, se algo der errado, a API responde 500 em vez de deixar o cliente esperando.

```
export const readAllUsers = async (req: Request, res: Response) => {
  try {
    const users = await UserRepository.findAllUsers(); // espera o banco responder
    return res.status(200).json(users);
  } catch (error) {
    console.error(error);                              // mostra o erro no terminal
    return res.status(500).json({ message: "Erro ao buscar usuários." });
  }
};
```

2 · Entendendo o projeto

Para instalar e rodar, siga o `README.md`. No desafio, você vai mexer em `prisma/schema.prisma` e `src/routes.ts` e criar dois arquivos: `src/repositories/PlantaRepository.ts` e `src/controllers/PlantaController.ts`.

As camadas

routes.ts qual função chamar? → **Controller** lê, valida e escolhe o status → **Repository** a única camada que usa o Prisma → **Banco** PostgreSQL

Camada	Faz	Não faz
routes	Diz que `POST /plantas` chama `createPlanta`	Nenhuma lógica ou validação
controller	Lê `req`, valida, decide o status, monta a resposta	Não usa o `prisma` diretamente
repository	Faz as consultas com o Prisma	Não conhece `req` nem `res`, não decide status

Lendo o CRUD de usuário

O model, em `prisma/schema.prisma`: cada model vira uma tabela, e cada linha, uma coluna.

```
model User {
  id        Int      @id @default(autoincrement())  // chave primária: 1, 2, 3...
  nome      String
  conta     Conta                                   // enum: CPF ou CNPJ
  documento String   @unique                        // não pode repetir
  createdAt DateTime @default(now())                // preenchido na criação
  updatedAt DateTime @updatedAt                     // atualizado a cada edição
}
```

As rotas, em `src/routes.ts`: a URL é o recurso no plural, e o que muda é o método.

```
routes.post("/users", UserController.createUser);
routes.get("/users", UserController.readAllUsers);
routes.get("/users/:id", UserController.readUserById);
routes.patch("/users/:id", UserController.updateUser);
routes.delete("/users/:id", UserController.deleteUser);
```

O repository: funções pequenas, uma consulta cada. `Prisma.UserCreateInput` é um tipo gerado a partir do model, e o TypeScript avisa se faltar campo ou o tipo estiver errado.

```
export const findUserById = async (id: number) => {
  return await prisma.user.findUnique({
    where: { id: id },
  });
};
```

O controller: quase todo handler segue **ler** → **validar** → **chamar o repository** → **responder**. Cada problema tem seu `return` com um status, e o caminho feliz fica no final.

```
export const createUser = async (req: Request, res: Response) => {
  try {
    const { nome, conta, documento } = req.body;           // ① lê

    if (!nome || !conta || !documento) {                   // ② valida
      return res.status(400).json({ message: "Preencha nome, conta e documento." });
    }

    const user = await UserRepository.createUser({         // ③ grava
      nome: nome, conta: conta, documento: documento,
    });

    return res.status(201).json(user);                     // ④ responde
  } catch (error) {
    console.error(error);
    return res.status(500).json({ message: "Erro ao criar usuário." });
  }
};
```

> 💡 **Experimente antes de codar**
> Chame todas as rotas de `/users` no Thunder Client, incluindo os erros: um id que não existe, um corpo vazio, um cadastro sem a conta. Ver as respostas ajuda muito a entender o código.

3 · Passo a passo

Os campos, as rotas e as regras estão no `DESAFIO.md`. Siga a ordem abaixo e faça um commit ao fim de cada passo.

PASSO 1 O model e o enum

Em `prisma/schema.prisma`, crie o enum `Luminosidade` e o model `Planta`. A sintaxe, com outro tema:

```
enum Tamanho {
  PEQUENO
  MEDIO
  GRANDE
}

model Camiseta {
  id        Int      @id @default(autoincrement())
  cor       String
  tamanho   Tamanho                                // usa o enum como tipo
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

☐ Os nomes dos campos estão exatamente como no enunciado (`regaACadaDias`, com as maiúsculas no lugar)?

☐ `preco` é `Int`, e não `Float`? Em centavos, as contas com dinheiro são exatas (em JavaScript, `0.1 + 0.2` dá `0.30000000000000004`).

☐ Tem `createdAt` e `updatedAt`, como no `User`?

PASSO 2 A migration

```
yarn migration
# Enter a name for the new migration: › create_planta_model
```

A migration é um arquivo SQL em `prisma/migrations/` que descreve a mudança no banco. Ela também atualiza os tipos do Prisma Client: depois dela, `prisma.planta` aparece no autocompletar. Confira com `yarn studio`.

> 🚫 **Errou o model? Não edite a migration**
> Corrija o `schema.prisma` e rode `yarn migration` de novo, com um nome como `fix_planta_preco`. As migrations vão para o git: são o "histórico de commits" do banco.

PASSO 3 O repository

Crie `src/repositories/PlantaRepository.ts` usando o `UserRepository.ts` como modelo. Use `Prisma.PlantaCreateInput` e `Prisma.PlantaUpdateInput` como tipo do `data`.

Função	O que faz	Prisma
`createPlanta(data)`	Cria uma planta	`create`
`findAllPlantas(filtros)`	Lista, aplicando os filtros que vierem	`findMany` + `where`
`findPlantaById(id)`	Busca uma planta ou devolve `null`	`findUnique`
`updatePlanta(id, data)`	Edita uma planta	`update`
`deletePlanta(id)`	Remove uma planta	`delete`
`venderPlanta(id, quantidade)`	Diminui o estoque (passo 7)	`update`
`calcularValorDoEstoque()`	Soma o valor do estoque (passo 8)	`findMany`

PASSO 4 O controller: validações

Crie `src/controllers/PlantaController.ts`. É o passo com mais detalhes.

Convertendo e validando o id. Se repetir em várias funções, vale criar uma função auxiliar no topo do arquivo.

```
const id = Number(req.params.id);     // "3" vira 3; "abc" vira NaN
if (!Number.isInteger(id) || id <= 0) {
  return res.status(400).json({ message: "O id precisa ser um número inteiro positivo." });
}
```

> **⚠ A armadilha mais importante:** `!campo`
>
> No `UserController`, `!nome || !conta` funciona porque todos os campos são textos. Nas plantas, quebra:
>
> | Valor recebido | `!valor` | É válido na planta? |
> |---|---|---|
> | `undefined` (não enviado) | true | ❌ Precisa recusar |
> | `0` (estoque zero) | true | ✅ Planta esgotada é válida |
> | `false` (petFriendly) | true | ✅ Planta tóxica para pets é válida |
>
> Para saber se um campo foi enviado, compare com `undefined`: `if (estoque === undefined)`.

Validando tipos e regras. Depois de garantir que o campo veio, confira o tipo e o valor:

Para validar...	Use
Texto não vazio	`typeof nome === "string" && nome.trim() !== ""`
Inteiro (recusa `"8990"`, `89.9` e `NaN`)	`Number.isInteger(preco)`
Booleano	`typeof petFriendly === "boolean"`

Valor do enum	`Object.values(Luminosidade).includes(luminosidade)`, com `import { Luminosidade } from "@prisma/client"`

Combinando com as regras: `!Number.isInteger(preco) || preco <= 0` significa "preço inválido". Escreva uma checagem para cada regra, cada uma com uma mensagem clara.

POST x PATCH. No cadastro, todos os campos são obrigatórios; na edição, nenhum é, mas os que vierem passam pelas mesmas regras. Uma função que valida só os campos enviados (pulando os `undefined`) serve para os dois. No POST, antes dela, confira se nenhum campo está faltando.

404 antes de editar ou deletar. Busque a planta primeiro e responda 404 se não existir. Sem isso, o erro do Prisma cai no `catch` e vira um 500 errado.

Não passe `req.body` direto para o banco. O cliente poderia mandar campos que você não espera, como `id` ou `createdAt`.

```
// ✅ você escolhe exatamente o que vai para o banco
const { nome, especie, preco, estoque, luminosidade, petFriendly, regaACadaDias } = req.body;
await PlantaRepository.createPlanta({ nome, especie, preco, estoque, luminosidade, petFriendly,
regaACadaDias });
```

PASSO 5 As rotas

Importe o controller e registre as rotas em `src/routes.ts`. O Express usa a **primeira** rota que combinar, então rotas fixas vêm antes das que têm parâmetro:

```
routes.get("/plantas/esgotadas", PlantaController.readPlantasEsgotadas);  // ✅ antes
routes.get("/plantas/:id", PlantaController.readPlantaById);
```

Na ordem inversa, `GET /plantas/esgotadas` chama `readPlantaById` com `id = "esgotadas"`.

PASSO 6 Filtros na listagem

`GET /plantas?luminosidade=SOMBRA&petFriendly=true` chega em `req.query`. Filtro não enviado é `undefined`, e no `where` do Prisma condições `undefined` são ignoradas: uma única consulta resolve todas as combinações.

```
// exemplo com usuários: se nome for undefined, lista todo mundo
prisma.user.findMany({
  where: { nome: nome },
});
```

- **Converta o booleano:** `petFriendly === "true"`, sem transformar um filtro não enviado em `false`.
- **Valide:** `?luminosidade=LUA` ou `?petFriendly=talvez` retornam 400.
- **Filtro sem resultado não é erro:** é 200 com `[]`, não 404.

> 💡 **Para os bônus**
> Pesquise na documentação do Prisma por `contains` com `mode: "insensitive"` (busca) e por `orderBy` (ordenação).

PASSO 7 A venda

1. Valide o id → 400.
2. Valide a `quantidade`: inteiro maior que zero → 400.
3. Busque a planta → 404.
4. Confira `planta.estoque >= quantidade` → senão 400, sem alterar nada. Para mensagens com variáveis, use crase: `` `Temos só ${planta.estoque} vasos` ``.
5. Diminua o estoque. O Prisma faz a conta: `data: { estoque: { decrement: quantidade } }`.
6. Responda 200 com a planta atualizada.

> 📘 **Para pensar (não é obrigatório)**
> E se duas pessoas comprarem o último vaso ao mesmo tempo? As duas podem passar pelo item 4 antes de qualquer uma chegar ao 5. Pesquise sobre `updateMany` com condição no `where`. Se resolver, conte no documento da API.

PASSO 8 O valor total do estoque

Para cada planta, `preco × estoque`; depois, some tudo.

Planta	preco	estoque	preco × estoque
Jiboia	3490	10	34900
Calathea	7990	2	15980
Total		**12 vasos**	**50880 (R$ 508,80)**

O `aggregate` do Prisma soma uma coluna, mas não uma multiplicação de colunas. Busque as plantas (só `preco` e `estoque`, com `select`) e some no código, com `for` ou `reduce`. Com a loja vazia, a resposta é 200 com os valores zerados.

4 · Testes

Crie uma coleção "Folhagem & Cia" no Thunder Client com uma requisição por rota, corpo em JSON. A maior parte da avaliação está nos casos de erro.

```
{
  "nome": "Costela-de-adão",
  "especie": "Monstera deliciosa",
  "preco": 8990,
  "estoque": 12,
  "luminosidade": "MEIA_SOMBRA",
  "petFriendly": false,
  "regaACadaDias": 7
}
```

Teste	Esperado
Cadastrar uma planta válida, inclusive com `estoque: 0` e `petFriendly: false`	201
Cadastrar sem um campo, com `preco: 0`, `estoque: -1`, `regaACadaDias: 0`, `preco: "8990"` ou `luminosidade: "LUA"`	400
Listar todas as plantas	200 + array
Buscar um id que existe / que não existe / `abc`	200 / 404 / 400
Editar só o preço (o resto não muda) / com valor inválido / id inexistente	200 / 400 / 404
Filtrar por luminosidade, por petFriendly e pelos dois / filtro inválido	200 / 400
Vender o que tem / mais do que tem (o estoque não muda) / `quantidade: 0` / id inexistente	200 / 400 / 400 / 404
Valor total (confira a conta na mão)	200
Deletar / buscar a planta deletada / deletar de novo	204 / 404 / 404

5 · Armadilhas comuns

Para erros de instalação e Docker, veja a tabela de problemas comuns do `README.md`. Estes são os erros de código que mais aparecem:

Erro	Como aparece	Como evitar
Validar com `!campo`	Estoque 0 ou `petFriendly: false` são recusados	`campo === undefined` (passo 4)
404 virando 500	Editar ou deletar id inexistente responde 500	Busque antes e responda 404
`catch` respondendo 400	Erro do servidor tratado como erro do cliente	No `catch` genérico, 500
Venda deixa estoque negativo	Vender 10 quando tem 3 funciona	Cheque `estoque >= quantidade`
Filtro como string	`?petFriendly=true` dá erro no Prisma	Converta `"true"` para `true`
Id sem conversão	Prisma reclama de string onde esperava Int	`Number(req.params.id)` e valide
Lista vazia como 404	Filtro sem resultado responde 404	Lista vazia é 200 com `[]`
`"${variavel}"` com aspas	A mensagem mostra o texto `${variavel}`	Use crase
Esquecer o `return`	`Cannot set headers after they are sent`	Todo `res.status(...)` dentro de `if` leva `return`
Esquecer o `await`	Resposta `{}` ou `if (!planta)` que nunca é verdadeiro	Toda chamada ao repository leva `await`
`req.body` direto no banco	O cliente consegue mandar `id`, `createdAt` …	Desestruture e passe campo a campo
Nomes diferentes do enunciado	`price`, `stock`, `/plants`	Use exatamente os nomes do `DESAFIO.md`
`Cannot find module '@controllers/...'`	A API não sobe	O nome no `import` precisa bater com o arquivo (letras e o `s` no final)
`PlantaController is not defined`	A API não sobe	Importe o controller no `routes.ts`
`Property 'planta' does not exist`	Erro de tipo no repository	`yarn migration` e **Restart TS Server** no VS Code

6 · Antes de entregar

Os entregáveis e os critérios de avaliação estão no `DESAFIO.md`. Confira:

☐ O projeto sobe do zero com `docker compose up -d --build` e `yarn migration`
☐ A migration da `Planta` está commitada em `prisma/migrations`
☐ Todos os testes da parte 4 passam
☐ Os nomes de campos e rotas estão exatamente como no enunciado
☐ `yarn typecheck` termina sem erros
☐ Não há `console.log` de teste, imports sem uso ou código comentado (o `console.error` do `catch` pode ficar)
☐ O `.env` não foi commitado
☐ O documento da API está completo e bate com o que a API faz

Nomes

O quê	Padrão	Exemplo
Variáveis e funções	camelCase	`plantaAtualizada`, `findPlantaById`
Arquivos de controller e repository	PascalCase + sufixo	`PlantaController.ts`
Rotas	minúsculas, plural, hífen	`/plantas`, `/valor-total`
Valores de enum	MAIÚSCULAS com `_`	`MEIA_SOMBRA`

Commits

Um commit por passo concluído, com mensagens que dizem o que mudou. Evite "ajustes", "agora vai" ou "final final 2": o histórico também é avaliado.

```
git commit -m "Cria model Planta e enum Luminosidade"
git commit -m "Adiciona validações de preço, estoque e luminosidade"
git commit -m "Implementa a rota de venda"
```

Travou? Como pedir ajuda

Pedir ajuda não tira ponto. Antes, tente uns 20 minutos: leia o erro com calma, veja os logs (`docker compose logs -f server`), compare com o CRUD de usuário e releia a parte 5. Ao pedir ajuda, mande o que queria que acontecesse, o que aconteceu (erro completo ou print), o que já tentou e o trecho de código.

Documentações: `prisma.io/docs`, `expressjs.com/pt-br` e `developer.mozilla.org` (status HTTP).

7 · Glossário

Termo	Significado
API	O programa que recebe requisições e devolve respostas. O "garçom" entre o cliente e o banco.
Rota (endpoint)	Método + URL que a API atende, como `GET /plantas`.
Requisição / resposta	O pedido do cliente (método, URL, params, query, body) e o que a API devolve (status + JSON).
Params / query / body	Parte variável da URL (`req.params`), o que vem depois do `?` (`req.query`) e os dados enviados (`req.body`).
Express	Biblioteca do Node.js para criar servidores e rotas HTTP.
Prisma (ORM)	Ferramenta para falar com o banco usando TypeScript em vez de SQL.
Model	A definição de uma tabela no `schema.prisma`.
Enum	Tipo que só aceita uma lista fechada de valores, como `SOMBRA | MEIA_SOMBRA | SOL_PLENO`.
Migration	Arquivo que registra uma mudança na estrutura do banco.
Controller / repository	A camada que valida e responde / a camada que consulta o banco.
Docker / container	Roda a API e o banco em ambientes isolados, iguais em qualquer máquina.
Seed	Script que preenche o banco com dados de exemplo.

> 🌿 **Boa sorte e boas regas!**
> Faça com calma, teste cada passo e capriche nos casos de erro. É neles que uma API mostra que foi bem feita.